

Алгоритмы и структуры данных

Лекция 3. Двоичные деревья

Ключарёв Пётр Георгиевич
pgkl@yandex.ru
twitter.com/klyucharev

Кафедра ИУ8 «Информационная безопасность»
МГТУ им. Н.Э. Баумана

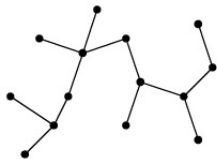
2013 г.

Дерево

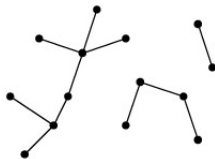
Дерево

Дерево (без выделенного корня) — это связный ациклический неориентированный граф.

Если убрать требование связности, получится *лес* (*forest*). Лес состоит из деревьев (являющихся его связными компонентами).



(a)



(b)



(c)

a — дерево; b — лес; c — лесом не является (и деревом тоже).

Корневые деревья

Корневое дерево

Дерево с корнем, или *корневое дерево* — дерево, в котором одна из вершин выделена. Выделенную вершину будем называть *корнем*. Вершины корневого дерева называются также *узлами* (node).

Предки и потомки

Пусть x — произвольная вершина корневого дерева с корнем в вершине r . Существует единственный путь из r в x . Все вершины, находящиеся на нем, назовем *предками* (ancestors) вершины x . Если y является предком x , то x называется *потомком* (descendant) y .

Поддерево

Для каждой вершины x можно рассмотреть дерево, состоящее из всех потомков x , в котором x считается корнем. Оно называется *поддеревом с корнем в x*

Корневые деревья

Родственные отношения

Если (y, x) — последнее ребро на пути из корня в x , то y называется *родителем* (parent) x , а x называется *ребёнком* y .

Корень является единственной вершиной, у которой нет родителя.

Вершины, имеющие общего родителя, называют *братьями*.

Вершина корневого дерева, не имеющая детей, называется *листом* (leaf).

Вершины, имеющие детей, называются *внутренними* (internal) вершинами.

Корневые деревья

Параметры

Число детей у вершины корневого дерева называется её *степенью* (degree).

Отметим, что для всех вершин, кроме корня, степень на единицу меньше степени той же вершины в том же дереве, если рассматривать дерево как неориентированный граф (поскольку тогда надо учитывать и ребро, идущее вверх).

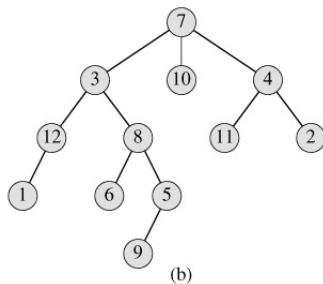
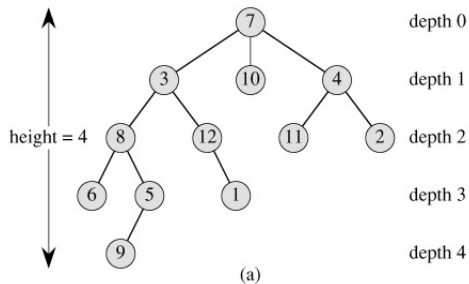
Длина пути от корня до произвольной вершины x называется *глубиной* (depth) вершины x .

Максимальная глубина вершин дерева называется *высотой* (height) дерева.

Дерево с порядком на детях

Деревом с порядком на детях (ordered tree) называется корневое дерево, для каждой вершины множество её детей упорядочено.

Корневые деревья



Это одно и то же корневое дерево, но с разным порядком на детях.

Двоичное дерево

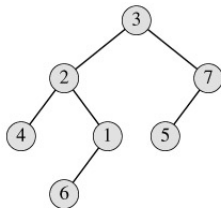
Двоичное дерево (binary tree)

Двоичное дерево — это конечный набор вершин, который либо пуст, либо разбит на три непересекающиеся части:

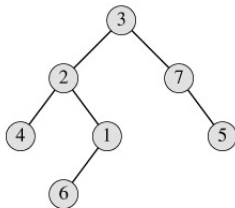
- вершину, называемую *корнем* (root);
- двоичное дерево, называемое *левым поддеревом* корня;
- двоичное дерево, называемое *правым поддеревом* корня.

Двоичное дерево, не содержащее вершин, будем называть *пустым* и обозначать *nil*. Если левое поддерево непусто, то его корень называется *левым ребёнком* корня всего дерева; *правый ребёнок* определяется аналогично.

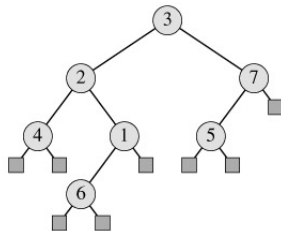
Двоичные деревья



(a)



(b)



(c)

k -ичное дерево

Позиционное дерево

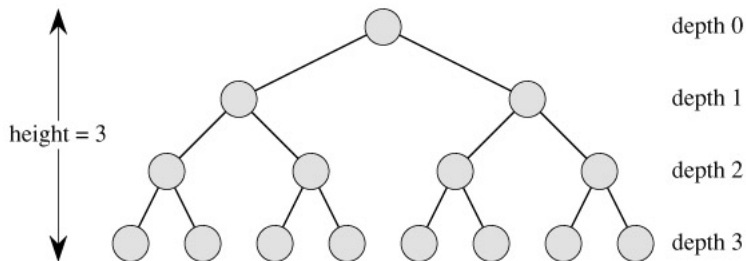
Позиционное дерево — это корневое дерево, в котором дети любой вершины пронумерованы — помечены различными натуральными числами. У каждой вершины есть вакансии для детей и конечное число этих вакансий заполнено.

k -ичное дерево

k -ичным деревом называется позиционное дерево, не имеющее вершин с номерами больше k .

Полным k -ичным деревом (complete k -ary tree) называется k -ичное дерево, в котором все листья имеют одинаковую глубину и все внутренние вершины имеют степень k .

Полное k -ичное дерево



Двоичное дерево поиска, как структура данных

Представление узла дерева

Узел представляет собой структуру со следующими полями:

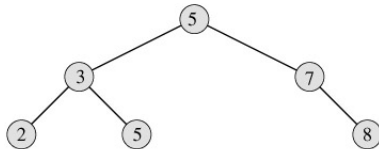
- Ключ *key*.
- Дополнительные данные.
- Указатель на левого потомка *left*.
- Указатель на правого потомка *right*.
- Указатель на родителя *p*.

Условие упорядоченности

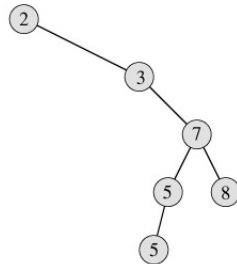
Ключи в двоичном дереве поиска хранятся так, чтобы соблюдалось *свойство упорядоченности*: Пусть x — произвольная вершина двоичного дерева поиска. Если вершина y находится в левом поддереве вершины x , то $y.key \leq x.key$.

Если y находится в правом поддереве x , то $y.key \geq x.key$.

Двоичные деревья поиска



(a)



(b)

Алгоритм вывода ключей в неубывающем порядке

Алгоритм 1 Вывод всех ключей дерева в неубывающем порядке

```
1: procedure INORDERTREEWALK( $x$ )  
2:   if  $x \neq nil$  then  
3:     InorderTreeWalk( $x.left$ )  
4:     print  $x.key$   
5:     InorderTreeWalk( $x.right$ )
```

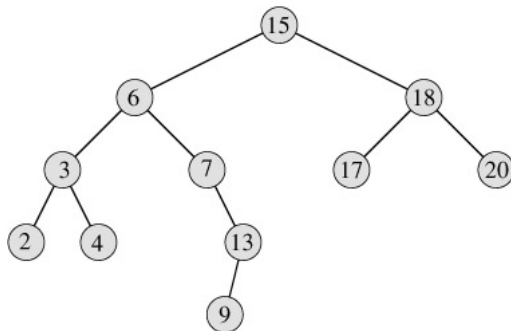
Алгоритм поиска в дереве

Алгоритм 2 Поиск в дереве

```
1: function TREESEARCH( $x, k$ )
2:   if  $x = nil$  или  $k = x.key$  then
3:     return  $x$ 
4:   if  $k < x.key$  then
5:     return TreeSearch( $x.left, k$ )
6:   else
7:     return TreeSearch( $x.right, k$ )
```

Глубина рекурсии не превосходит высоту дерева, поэтому, время поиска есть $O(h)$.

Двоичные деревья поиска



Алгоритм поиска в дереве

Алгоритм 3 Поиск в дереве (итеративный)

```
1: function TREESearchIterative( $x, k$ )
2:   while  $x \neq nil$  и  $k \neq x.key$  do
3:     if  $k < x.key$  then
4:        $x := x.left$ 
5:     else
6:        $x := x.right$ 
7:   return  $x$ 
```

Минимум и максимум в дереве

Алгоритм 4 Поиск минимума

```
1: function TREEMINIMUM( $x$ )  
2:   while  $x.left \neq nil$  do  
3:      $x := x.left$   
4:   return  $x$ 
```

Алгоритм 5 Поиск максимума

```
1: function TREEMAXIMUM( $x$ )  
2:   while  $x.right \neq nil$  do  
3:      $x := x.right$   
4:   return  $x$ 
```

Элемент, следующий за данным

Алгоритм 6 Поиск элемента, следующего за данным

```
1: function TREESUCCESSOR( $x$ )
2:   if  $x.right \neq nil$  then
3:     return  $TreeMinimum(x.right)$ 
4:    $y := x.p$ 
5:   while  $y \neq nil$  и  $x = y.right$  do
6:      $x := y$ 
7:      $y := y.p$ 
8:   return  $y$ 
```

Сложность $O(h)$.

Добавление элемента в дерево

Алгоритм 7 Добавление элемента в дерево

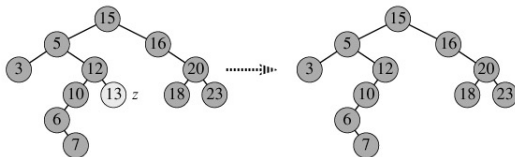
```
1: procedure TREEINSERT( $T, z$ )
2:    $y := nil$ 
3:    $x := T.root$ 
4:   while  $x \neq nil$  do
5:      $y := x$ 
6:     if  $z.key < x.key$  then
7:        $x := x.left$ 
8:     else
9:        $x := x.right$ 
10:   $z.p := y$ 
11:  if  $y = nil$  then
12:     $T.root := z$ 
13:  else if  $z.key < y.key$  then
14:     $y.left := z$ 
15:  else
16:     $y.right := z$ 
```

Некоторые свойства

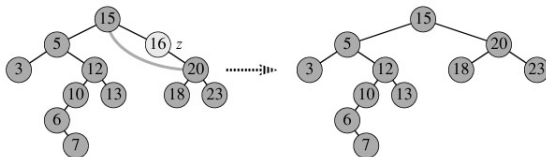
Теорема

Если вершина двоичного дерева поиска имеет двоих детей, то следующая за ней вершина не имеет левого ребенка, а предшествующая — правого.

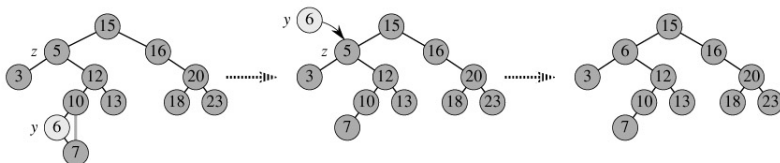
Удаление элемента z из дерева



(a)



(b)



(c)

Удаление элемента из дерева

Алгоритм 8 Удаление элемента из дерева

```
1: procedure TREEDELETE( $T, z$ )
2:   if  $z.left = nil$  или  $z.right = nil$  then
3:      $y := z$ 
4:   else
5:      $y := TreeSuccessor(z)$ 
6:   if  $y.left \neq nil$  then  $x := y.left$ 
7:   else  $x := y.right$ 
8:   if  $x \neq nil$  then  $x.p := y.p$ 
9:   if  $y.p = nil$  then  $T.root := x$ 
10:  else if  $y = y.p.left$  then
11:     $y.p.left := x$ 
12:  else
13:     $y.p.right := x$ 
14:  if  $y \neq z$  then  $z.key := y.key$ 
15:  return  $y$ 
```

Сложность

Сложность основных операций над деревом (поиска, удаления, вставки и др.): $O(h)$.

Но как h связано с n ?

Упражнения

- 1 Найти высоту полного k -ичного дерева с n вершинами.
- 2 Найти число внутренних вершин и число листьев у полного k -ичного дерева высоты h .
- 3 Показать, что высота двоичного дерева с n вершинами не меньше $\lfloor \log_2 n \rfloor$
- 4 Доказать, что в любом двоичном дереве число вершин степени два на единицу меньше числа листьев.
- 5 Пусть поиск ключа в двоичном дереве завершается в листе. Рассмотрим три множества: A (элементы слева от пути поиска), B (элементы на пути) и C (справа от пути). Профессор утверждает, что для любых трёх ключей $a \in A$, $b \in B$ и $c \in C$ верно $a \leq b \leq c$. Покажите, что он неправ, и приведите контрпример минимально возможного размера.

И еще упражнения...

- 1 Построить нерекурсивный алгоритм, выводящий ключи в двоичном дереве поиска в неубывающем порядке.
- 2 Напишите рекурсивный вариант процедуры *TreeInsert*.